A Model-Driven Approach for Generating Electrochemical Reactor Simulation Models

James Mascarenhas
Queen's University at Kingston
Kingston, Ontario, Canada
19jpm@queensu.ca

Jihyeon Park
Queen's University at Kingston
Kingston, Ontario, Canada
17jp53@queensu.ca

Keywords

electrochemistry, eCherry, electrochemical reactor, model-driven software development, software modelling

ACM Reference Format:
James Mascarenhas and Jihyeon Park. 2026. A Model-Driven Approach for Generating Electrochemical Reactor Simulation Models. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX).* ACM, New York, NY, USA, 2 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Electrochemical reactors are widely used in applications such as water electrolysis, fuel cells, and other energy conversion and chemical processing systems. Modeling these reactors is important because simulation helps researchers study system behavior, evaluate operating conditions, and explore design alternatives before physical implementation.

Electrochemical reactor simulation frameworks such as *eCherry* allow for the modeling of these electrochemical systems using the Modelica language [4]. Frameworks like these enable researchers and engineers to construct modular simulation models by assembling reusable components representing electrodes, electrolytes, separators, and flow channels.

1.1 Problem

However, constructing electrochemical reactor simulations requires manually assembling multiple low-level Modelica components and configuring many simulation parameters. This process can be quite complex and time-consuming for domain experts who are primarily trained in electrochemistry rather than simulation modeling. As a result, building and modifying electrochemical reactor models requires significant effort and familiarity with the underlying modeling framework.

1.2 Proposed Solution

Model-Driven engineering is used for similar problems in software development where raising the level of abstraction used to describe systems, and then automatically generating executable artifacts from high-level models [5, 6]. In this solution we will explore a model-driven approach of how to build electrochemical reactor simulations by introducing a domain-specific language (DSL) that allows users to describe reactor configurations at a higher level of abstraction. Models specified in the DSL will then be transformed into *eCherry* (an open-source chemistry library) compatible Modelica simulation models, generating executable artifacts automatically.

The goal of this approach is to address the limits of other electrochemistry models that demonstrate a lack of reusability and accessibility, and simplify reactor modeling. We plan to achieve this by eliminating the necessity for manual low-level assembly of Modelica components. This provides the novelty of the proposed approach by introducing a model-driven workflow for electrochemical reactor modeling. As a result, building simulation models should become more efficient for reactor analysis and design exploration.

2 Proposed Methods

The proposed method follows model-driven software development workflow in which high-level reactor descriptions are automatically translated into executable simulation models compatible with the *eCherry* framework [7]. This approach introduces a domain-specific modeling layer that abstracts away low-level Modelica component configuration while preserving compatibility with the underlying simulation library.

The modeling pipeline will consist of four stages:

- DSL Reactor Model
- Metamodel-based representation
- Model Transformation
- Automated Code Generation compatible with the eCherry library

The users will describe their models using high-level domain concepts rather than manually assembling low-level Modelica structures. Based on the user input, the system will construct a metamodel-based representation and apply model transformation rules that convert the high-level designs into the lower-level eCherry compatible simulation models. Once the transformation is completed, the corresponding Modelica code will be automatically generated.

2.1 Metamodel in electrochemistry

The first step of the proposed approach is to define a metamodel that captures the structure of electrochemical reactor systems [7]. The metamodel must represent the key components and relationships that occur in these electrochemical simulations.

Entities represented in the metamodel, will include; reactors, electrodes, electrolytes, separators, flow channels, and operating

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, Woodstock, NY
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

conditions such as temperature and flow rate. The job of the metamodel is then to define the relationships between these and establish the structural foundation for the modeling language.

This approach focuses on capturing the structural behavior of reactor systems instead of predicting or planning reactions. The role that the metamodel will play is to represent reactor configuration in a form that can be systematically transformed into executable eCherry Modelica simulation model.

2.2 Designing A Domain-Specific Language (DSL)

Designing a domain-specific language for electrochemistry is a central part of this approach [1]. The purpose of designing a domain-specific language is to allow users to describe electrochemical reactor systems at a higher level of abstraction rather than directly constructing them through low level. Instead of manually defining detailed component connections and implementation-specific structures, users will specify the reactor system using domain knowledge.

For example, a simplified reactor specification or input for domain-specific language may appear as:

```
reactor Electrolyzer {
    electrode_area = 10 cm^2
    electrolyte = KOH
    temperature = 298K
}
```

The DSL would behave as an intermediate modeling layer between the user and the code that is generated to be compatible with eCherry. It captures the essential structure and operating conditions of the reactor in a concise and readable form, while hiding unnecessary implementation complexity. This higher-level description can then be parsed and transformed automatically into an executable electrochemistry model [3]. As a result, the domain-specific language would improve usability, reduces manual coding effort, and allow effective exploration of reactor systems.

2.3 Model Transformation

A model transformation will be responsible for converting the DSL models into their eCherry compatible Modelica simulation model counterparts. This process will involve parsing the DSL specification, constructing an internal representation based on the metamodel, and applying a set of transformation rules that will map DSL constructs to corresponding eCherry Modelica components. This transformation is the most important part of the method in terms of ensuring that the high-level reactor descriptions are correctly translated into accurate executable simulation structures.

2.4 Generated Artifacts

The output of this model transformation will be executable Modelica simulation models [2]. These generated models can then be executed with a standard Modelica simulation environment. The automation of this process is meant to enable users to quickly define, modify and test reactor configuration without the manual assembly of low-level Modelica models.

3 Implementation Plan

To evaluate this model-driven approach we will implement a prototype tool that automatically generates eCherry compatible Modelica simulation models from high-level reactor specifications. This prototype will demonstrate the full modeling pipeline, including DSL parsing, model transformation, and automatic code generation.

3.1 Tools and Technologies

The prototype system will likely be implemented using Python to support DSL parsing, model transformation and code generation. And the generated Modelica models will target the eCherry library and be executable in standard Modelica simulation environments.

4 Expected Results

For evaluation, we will use several known reactor configurations that have been constructed manually to serve as baseline models. We will then create those same configurations using this proposed system to enable comparison between the manually constructed and automatically generated models in terms of simulation accuracy and development effort (measured using time taken for development).

The main expected outcome is that users will be able to define reactor systems at a higher level of abstraction and automatically generate code that is compatible with the eCherry library. In that way, it becomes possible to observe the simulation in electrochemical environments, and will be highly accessible and reusable. More specifically, we hypothesize that the automatically generated models will reproduce the behavior of the manually constructed ones with minimal difference in simulation outputs, at a fraction of the effort or time.

4.1 Potential Challenges

Designing a domain-specific model may be the most challenging part since it requires outstanding domain-specific knowledge in a specific field. While designing domain-specific languages, what should be checked are specificity and generalizability. Furthermore, the model transformation that maps the DSL models to the Modelica simulation models must cover a large variety of reactor configurations and component relationships. Ensuring that these mappings are complete and accurate is essential, as any errors in this process will lead to incorrect simulation behavior.

References

[1] Martin Fowler. 2010. *Domain-Specific Languages.* Addison-Wesley.
[2] Peter Fritzson. 2015. *Principles of Object-Oriented Modeling and Simulation with Modelica 3.3: A Cyber-Physical Approach.* Wiley.
[3] Steven Kelly and Juha-Pekka Tolvanen. 2008. *Domain-Specific Modeling: Enabling Full Code Generation.* Wiley.
[4] J. F. Pyschik, K. M. Ebeling, G. I. Prokopou, M. J. Rix, J. R. Seidenberg, M. B. Kleiner, and D. Bongartz. 2025. eCherry: A Modelica Library for Modular Dynamic Modelling of Electrochemical Reactors. *Electrochemical Science Advances* 5, 5 (2025), e202400030.
[5] Douglas C. Schmidt. 2006. Model-Driven Engineering. *Computer* 39, 2 (2006), 25–31.
[6] Bran Selic. 2003. The Pragmatics of Model-Driven Development. *IEEE Software* 20, 5 (2003), 19–25.
[7] Thomas Stahl, Markus Völter, and Krzysztof Czarnecki. 2006. *Model-Driven Software Development: Technology, Engineering, Management.* Wiley.